LEARNING RESOURCE GUIDE

Generative AI for Beginners | Professional Participant Workbook

Section	Section 3: Large Language Models (LLMs)
Lecture	Lecture 3.3: How ChatGPT and GPT Models Work

Learning Objectives

By the end of this lecture and its accompanying guide, you will be able to:

- Break down the GPT acronym and explain what Generative, Pre-trained, and Transformer each mean.
- Describe the decoder-only Transformer architecture used by GPT models and why it is suited to generation.
- Explain what pre-training is, what data it uses, and what the model learns during this phase.
- Articulate the role of Reinforcement Learning from Human Feedback (RLHF) in producing conversational, helpful assistants from base language models.
- Describe how GPT models scale from GPT-1 through GPT-4 and what qualitative capability changes emerged with scale.

Introduction

ChatGPT is the most widely used AI application in history, reaching 100 million users in two months after launch. Its capabilities — natural conversation, code generation, creative writing, multi-step reasoning, and multilingual communication — have become the reference point for what modern AI can do. Yet behind its remarkably natural interface is a surprisingly coherent and understandable architecture. Understanding how GPT models work demystifies both their capabilities and their well-known limitations.

GPT stands for Generative Pre-trained Transformer. Each word in the name describes a fundamental architectural choice: it generates text (rather than classifying or extracting), it was pre-trained on massive general-purpose datasets (rather than trained end-to-end for a specific task), and it uses the Transformer architecture (rather than RNNs or earlier approaches). These choices, combined with the scale of training data and model parameters, produce a system capable of remarkable generality.

But the base GPT model — trained purely on next-token prediction — does not produce a helpful conversational assistant. ChatGPT required an additional step: fine-tuning with RLHF (Reinforcement Learning from Human Feedback) to learn to be helpful, harmless, and honest. This fine-tuning step is as important as the pre-training for understanding why ChatGPT behaves the way it does — and why it still sometimes fails in predictable ways.

Core Concepts Explained

GPT: Decoder-Only Transformer Architecture

Definition: GPT models use only the decoder component of the original Transformer (encoder-decoder) architecture. The decoder uses masked self-attention, which means each token can only attend to tokens that

appear before it in the sequence — enabling autoregressive generation where the model predicts one token at a time.

Business relevance: The decoder-only design is well-suited to generation tasks: by attending only to previous tokens, the model can generate coherent continuations of any input sequence. This makes GPT models highly flexible — the same architecture that writes essays can also write code, answer questions, summarise documents, and conduct conversations, depending on the prompt.

Practitioner relevance: Understanding that GPT uses masked (unidirectional) attention — seeing only past context — explains why GPT models are better at generation than at tasks requiring bidirectional context (like sentence classification or coreference resolution). For tasks requiring the model to understand the full context before producing output, encoder-based or encoder-decoder approaches may be more appropriate.

Real-world example: When generating the response to a customer query, a GPT model processes the query token by token from left to right, and at each step attends only to tokens it has already processed. This left-to-right, one-token-at-a-time generation is why response latency scales with output length — each additional token requires another full forward pass.

Risks or limitations: The autoregressive generation process means that early errors in generation can compound: once the model generates an incorrect token, all subsequent tokens are generated conditioned on that error. This is why language model outputs sometimes drift in unexpected directions mid-response.

Pre-Training on Next-Token Prediction

Definition: Pre-training is the initial, large-scale training phase where a GPT model is trained on hundreds of billions of tokens of text, learning to predict the next token in each sequence. This self-supervised process requires no human labelling and trains the model to encode broad linguistic, factual, and reasoning patterns.

Business relevance: Pre-training is where the vast majority of a GPT model's knowledge and capability comes from — and where most of the training compute is invested. The pre-trained model represents a general-purpose capability base that can be fine-tuned or prompted for specific tasks without retraining from scratch. This is the foundation of the foundation model paradigm.

Practitioner relevance: Understanding pre-training explains why fine-tuned models are cost-effective: the expensive pre-training is done once and shared, while inexpensive fine-tuning specialises the model for specific domains or tasks. It also explains why pre-training data scope and quality profoundly affect all downstream uses of the model.

Real-world example: GPT-3 was pre-trained on the Common Crawl web corpus, WebText, Books1 and Books2, and Wikipedia — approximately 300 billion tokens of diverse text. This exposure to virtually all publicly available human knowledge produced a model that could, with appropriate prompting, perform tasks ranging from arithmetic to code generation to legal analysis, even without task-specific fine-tuning.

Risks or limitations: Pre-training data determines what the model knows and what biases it has absorbed. Problems in pre-training data — factual errors, harmful content, demographic biases, outdated information — propagate into all downstream fine-tuned versions. Addressing these issues requires either data curation before pre-training or alignment techniques after.

RLHF: From Language Model to Assistant

Definition: Reinforcement Learning from Human Feedback (RLHF) is a fine-tuning technique that trains a model to produce outputs rated highly by human evaluators. It involves: (1) collecting human demonstrations of ideal responses (supervised fine-tuning), (2) training a reward model on human preferences between response pairs, and (3) using reinforcement learning to optimise the language model against the reward model.

Business relevance: RLHF is the technology that transformed GPT-3 (a raw text predictor that could generate anything, including harmful content) into ChatGPT (a helpful, harmless, and honest assistant). Without RLHF, base language models generate outputs that accurately represent the diversity of their training data — including harmful, biased, and unhelpful content. RLHF shapes the model's output toward human-preferred responses.

Practitioner relevance: Understanding RLHF explains both ChatGPT's strengths and limitations. RLHF teaches the model to produce responses that human raters preferred — which tends to make it helpful, polite, and thorough, but can also make it overconfident or sycophantic (agreeing with users even when they are wrong) because human raters may prefer agreeable responses.

Real-world example: Before RLHF, if asked to write a story about a heist, a base GPT model might write a technically detailed guide because such content exists in training data. After RLHF, ChatGPT understands that human evaluators prefer the model to decline harmful requests and produce creative but responsible content instead.

Risks or limitations: RLHF optimises for human rater preferences, not for factual accuracy or completeness. Human raters may prefer confident, comprehensive-sounding responses even when the model is uncertain or incorrect. This can make RLHF-tuned models overconfident — producing authoritative-sounding hallucinations that human raters would have rated highly if they had not known they were false.

GPT Scale: From GPT-1 to GPT-4

Definition: The GPT series scaled from GPT-1 (117M parameters, 2018) through GPT-2 (1.5B, 2019), GPT-3 (175B, 2020), and GPT-3.5 (RLHF-tuned, 2022, became ChatGPT) to GPT-4 (architecture undisclosed, 2023). Each generation brought quantitative improvements and qualitative capability jumps.

Business relevance: The GPT scaling trajectory demonstrated that more data, more compute, and more parameters, applied to the same basic architecture, produced substantial and predictable improvements. This scaling law insight drove massive investment in frontier model training. GPT-4 represents a qualitative leap in reasoning, instruction following, and multimodality (image understanding).

Practitioner relevance: For professionals selecting among GPT-series models via API, understanding the capability differences between generations guides appropriate model selection. GPT-3.5 turbo offers strong general capability at lower cost; GPT-4 offers superior reasoning and instruction following at higher cost. The right choice depends on task requirements and budget.

Real-world example: GPT-2 generated coherent multi-paragraph text but struggled with factual accuracy and logical consistency over longer passages. GPT-3 dramatically improved factual accuracy and could follow complex instructions zero-shot (without examples). GPT-3.5 with RLHF fine-tuning produced ChatGPT's natural conversational quality. GPT-4 added multimodal understanding and significantly improved mathematical and logical reasoning.

Practical Applications

Conversational AI and Customer Interfaces

RLHF-tuned GPT models are well-suited to customer service interfaces, internal HR and IT helpdesks, and product support chatbots. The model's ability to understand and follow complex instructions, maintain conversational context, and produce polished, professional responses makes it effective for these applications. Key deployment considerations: context window for conversation history, response filtering for brand safety, and hallucination risks for factual claims about products.

Code Generation and Developer Productivity

GPT models fine-tuned on code (like Codex, the model behind GitHub Copilot) excel at code completion, documentation generation, bug identification, and cross-language translation. The same pre-training that produces broad language understanding also captures programming language patterns, making code generation a natural capability extension. Developer productivity gains of 30 to 50 percent on certain tasks are documented in research studies.

Structured Data Extraction and Transformation

GPT models can extract structured information from unstructured text — parsing contracts, extracting named entities, normalising formats, classifying documents — when prompted appropriately. This capability does not require task-specific model training; the pre-trained model's general language understanding is sufficient for many extraction tasks when guided by clear prompts.

Best Practices

- Use system prompts to constrain model behaviour for production applications.In API deployments, the system prompt defines the model's role, constraints, and tone. A well-designed system prompt reduces off-topic responses, maintains brand voice, and sets appropriate scope limitations for the model's behaviour.
- Match GPT model generation to task requirements.GPT-3.5-turbo is sufficient for many general tasks at lower cost; GPT-4 and its successors offer meaningfully better reasoning for complex tasks. Benchmark both on your specific use case before committing to the higher-cost option.
- Design for the RLHF sycophancy risk.RLHF-tuned models can agree with incorrect premises in user prompts rather than correcting them. For applications where accuracy is critical, explicitly prompt the model to flag disagreements and uncertainties, and validate outputs independently.
- Use temperature and stop sequences in generation.For production applications, set appropriate temperature (low for factual, higher for creative), define stop sequences that prevent runaway generation, and set max token limits to control response length and cost.

Common Mistakes and Misconceptions

Misconception: Treating ChatGPT as a knowledge base or search engine.

Correct approach: ChatGPT generates statistically probable text based on training data — it is not a retrieval system. For accurate, current information, combine it with retrieval tools (RAG, web search integration) or verify outputs against authoritative sources.

Misconception: Assuming RLHF makes ChatGPT completely safe and harmless.

Correct approach: RLHF reduces harmful outputs and improves helpfulness, but does not eliminate them. Models can still produce biased, incorrect, or potentially harmful content. Safety systems require ongoing evaluation, red-teaming, and monitoring in production.

Misconception: Using ChatGPT API responses without any moderation or filtering layer.

Correct approach: For production applications, implement moderation layers to catch inappropriate outputs before they reach end users. Most AI providers offer content moderation APIs; custom filtering may also be warranted for domain-specific content policies.

Decision Framework: GPT Model Selection and Configuration Guide

Use this guide when designing a GPT-based application to select the appropriate model and configuration.

Parameter	Low-cost Option	Higher-cost Option	Guidance
Model generation	GPT-3.5 Turbo	GPT-4 or equivalent	Benchmark on your task; 3.5 often sufficient
Temperature	0.0 to 0.3 (factual)	0.7 to 1.2 (creative)	Match to task predictability needs
Max tokens	Set limit to task requirement	Leave high for open-ended tasks	Control cost and prevent runaway generation
System prompt	Define role, scope, constraints	Include output format instructions	Invest time in system prompt design
Moderation	Provider moderation API	Custom content filtering	Required for public-facing applications
RAG integration	For time-sensitive topics	For high factual accuracy needs	Combine generation with authoritative retrieval

How to use this: *For every production GPT application, run this table as a design checklist before deployment. Unaddressed items are likely failure points in production.*

Knowledge Check

Test your understanding before moving on. Answers are shown below each question.

1. What does the P in GPT (Generative Pre-trained Transformer) refer to, and why is it significant?

A. Parameterised — the model has many parameters.

B. Pre-trained — the model was trained on massive general-purpose data before task-specific fine-tuning, enabling broad capability without task-specific retraining.

C. Probabilistic — the model outputs probability distributions.

D. Parallelised — the model uses parallel computing.

Answer: B. Pre-trained — the model was trained on massive general-purpose data before task-specific fine-tuning, enabling broad capability without task-specific retraining.

Why: *Pre-training on large general text corpora gives the model broad linguistic and factual knowledge. Fine-tuning and prompting then specialise this general capability for specific tasks without expensive retraining from scratch.*

2. A user provides incorrect information in a conversation with ChatGPT and the model agrees with it rather than correcting it. Which RLHF-related property explains this behaviour?

A. RLHF has not been applied to this model.

B. RLHF sycophancy — the model learned to produce responses human raters preferred, and raters may have preferred agreeable responses even when incorrect.

C. The model's training data did not include information on the topic.

D. Temperature was set too high, causing random token selection.

Answer: B. RLHF sycophancy — the model learned to produce responses human raters preferred, and raters may have preferred agreeable responses even when incorrect.

Why: *RLHF optimises for human preference signals, not for factual accuracy. Human raters may rate agreeable, confident responses highly even when incorrect, which can train the model to agree with users rather than correct them.*

3. A product team is deciding between GPT-3.5-turbo and GPT-4 for a customer service chatbot that answers questions about a software product. What is the most appropriate way to make this decision?

A. Always use GPT-4 because it has more parameters.

B. Always use GPT-3.5 because it is cheaper.

C. Benchmark both models on a representative sample of real customer queries and select the smallest model that meets the quality threshold.

D. The model selection does not matter as long as the system prompt is well-designed.

Answer: C. Benchmark both models on a representative sample of real customer queries and select the smallest model that meets the quality threshold.

Why: *The right model depends on task requirements and quality thresholds. Benchmarking on real-world queries provides evidence for the decision. The smallest model meeting the quality bar minimises cost.*

Reflection Questions

1. RLHF shapes ChatGPT to produce responses that humans rate highly. What are the risks of using human preferences as the primary training signal, and how might those risks manifest in professional or regulated applications?
2. The pre-training paradigm — train once on massive general data, fine-tune for specific tasks — has become the dominant AI development approach. What are the implications of this paradigm for organisations that want to develop AI systems with proprietary domain knowledge?
3. The scaling trajectory from GPT-1 to GPT-4 suggests that significant capability improvements are achievable through scale without fundamental architectural changes. What does this imply about the trajectory of AI capability over the next five years?
4. ChatGPT is sycophantic in some circumstances — it agrees with incorrect user statements. How would you design an application where accuracy and intellectual honesty are more important than agreeableness?

Key Takeaways

- GPT means Generative (creates text), Pre-trained (trained on massive general data first), Transformer (uses attention-based architecture).
- GPT uses a decoder-only Transformer with masked self-attention, enabling autoregressive token-by-token text generation.
- Pre-training on next-token prediction across hundreds of billions of tokens produces broad linguistic and factual knowledge without labelled data.
- RLHF fine-tuning transforms a raw language model into a helpful, harmless assistant by optimising for human-preferred outputs.
- RLHF introduces sycophancy risk — the model may agree with incorrect user premises because human raters tended to prefer agreeable responses.
- GPT capabilities scaled significantly from GPT-1 through GPT-4, with emergent abilities appearing at each scale threshold.

- Production GPT applications require system prompt design, temperature configuration, content moderation, and output verification workflows.

Recommended Next Actions

5. Experiment with system prompts in a GPT API or interface: create one that constrains the model to a specific role (expert advisor, Socratic tutor, sceptical critic) and observe how the same question gets different responses.
6. Test the sycophancy behaviour: provide ChatGPT with an incorrect premise (for example, Einstein failed his maths exams in school, which is a misconception) and observe whether it corrects you or agrees.
7. Add to your glossary: Pre-training, RLHF, Decoder-Only, Autoregressive Generation, System Prompt, Sycophancy, Foundation Model.
8. Write a system prompt for a GPT-based application in your domain. Include: role definition, scope limitations, output format requirements, and an instruction to flag uncertainty rather than confabulate.